The AUT AT90USB1287 Lab board software library documentation

Version 1.0

March 2013.

Disclaimer

The use of the software library that this document describes is not guaranteed to be fit for any particular purpose and is used entirely at your own risk. The author does not take any responsibility for the consequences of it use. Neither is the code described guaranteed to be free of fault, nor is this document guaranteed to be accurate.

Table of Contents

Introduction

A microcontroller board that utilises the Atmel AT909USB1287 processor has been developed and built by the Electrical Engineering Technicians. These boards are fixed to the Lab benches in labs WS412 and WS413. The AT90USB1287 lab boards have a board number 111007. The boards communicate with a host computer via an Atmel Dragon ICE (In Circuit Emulator) that is enclosed within the board's box and connects to the computer via USB. The 111007 microcontroller board will be referred to as the *micro board* in this document.

Using the IAR Systems Embedded Workbench for AVR, a library of code modules have been developed by the author that provide a C programming interface to each of the peripheral devices on the micro board. The only exceptions are the processor's USB interface and the interface to the 20 pin external header. This library was developed for two reasons:

To enable a menu driven test program to be written that tests the hardware functionality of each of the peripherals on the board for board testing purposes.
To provide students and staff with an easy way of using the board's peripherals without having a detailed register level understanding of hardware interfaces.

The library code has been compiled into one library file called LibraryNDB.r90 and has been placed on the AUT network at the location:

K:\WELLESLEY COPY\DESIGN & CREATIVE TECHNOLOGIES\School of Engineering\Electrical & Electronic Engineering\Resources\IARLibraries\LabBoard\Lib

The library is simply a collection of C functions that have been compiled and placed into the library file. Any of the functions in the library can be used when it is added as one of the source files in a user's program. The user's program will also be called the *client* or *client code*.

The library file is in binary and cannot be read by a text editor. The library's name includes the letters 'NDB' to mean "No Debug Information". Practically, this means that when a program that uses the library is stepped through using the debugger, debugger does not attempt to step through the library's source code, but executes library functions completely each time they are called in a C program. The source code to the library is not available on the K: drive and not available for use.

Every function in the library has C prototype that is placed into a header file that has the file extension ".h". In general, each separate C source file that has been compiled and is part of the library has a separate header file and all of these header files are placed in the folder:

K:\WELLESLEY COPY\DESIGN & CREATIVE TECHNOLOGIES\School of Engineering\Electrical & Electronic Engineering\Resources\IARLibraries\LabBoard\Include

However, the library user does not need to know which specific header file to use, since simply including the one header file: "Labboard.h", will include all of the others necessary. Therefore every C program that uses the library will have the line:

```
#include <Labboard.h>
```

placed at the start of any C source file that uses function in the library.

This document will describe the functions in the library and give some examples of how to use them. The functions descriptions are divided into groups, where each group applies to a particular peripheral device, for example the LCD display, USART, or motor fan. Function prototypes for a

particular peripheral device will all appear in the one header file that is included in overall "Labboard.h" file. For every function in a header file, there is:

1. A C prototype.
2. A brief description of what the function does.
3. The arguments that the function accepts.
4. What parameter the function returns to the calling program.

Even without this document, the information in the various header files should be enough to use the library functions.

How to configure your IAR project to use the library will be described next, using a simple example.

Configuring an IAR project to use the library

There are four steps to configuring your project to use the library. These are:

Set the pre-processor include files search folder.
Set the stack space for your project to 255 bytes.
Set the IAR library used by your project to DLIB, not the default CLIB.;
Add the library file to your project's list of files.
Ensure that the debugger is set to the Dragon.

These steps will be described below.

Setting the include files search folder

After you have created a project in IAR workbench, the left hand column of the screen will display a list of files in the project. Right clicking on the project name will bring up a drop down menu where the top item is called "Options…". Click on it.

The left hand pane of the options window shows a list called "Category". Select the category "C/C++ Compiler" then scroll along the tabs on the top right hand side until the "Preprocessor" tab is found. Next, cut and paste the above include folder path into the pane labelled, "Additional include directories". This can be done by running Windows Explorer (not Internet Explorer) and finding the above folder on the Copy (K:) drive. Then right click on the file path pane at the top of the Explorer main window and select "Copy address as text".

Place the cursor back IAR workbench pane "Additional include directories" and press Ctrl+V or right click and select Paste and OK. This will mean that IAR Workbench will search this folder whenever it finds an include file enclosed by angle brackets <>.

Set the stack space for your project

On the Options window select the category General and then scroll along the top tabs to the System tab. In the Size (bytes) box put 255 or 0xFF.

Set the IAR library used to DLIB

Scroll back along the top tabs to the Library Configuration tab. From the Library drop box, select "Normal DLIB". Click OK on the bottom of the Options window to save and exit setting the project options.

Adding the library file to your project

Right click on the your project's name on the left hand column of the screen and select "Add". From the Add drop down menu, select "Files". An Explorer window will pop up. Select the K: (Copy) drive and browse to :

K:\WELLESLEY COPY\DESIGN & CREATIVE TECHNOLOGIES\School of Engineering\Electrical & Electronic Engineering\Resources\IARLibraries\AppBoard\Lib

The library file will not be shown by default. Notice that the menu drop box on the bottom right corner of the screen shows "Source files". Select "All files", then the library file will be displayed. Select the file LibraryNDB.r90, and click "Open". The file will appear in the list of source files for your project. This completes the project setup required to use the library.

Setting the debugger

To download and debug your program the debugger must be changed from the default which is a simulator to the Dragon ICE (In Circuit Emulator). From the Options windows, select debugger and from the Driver drop box, select "Dragon". If this is not done, then your program will appear to download and run, but it will not be running on the lab board, but only on the computer simulator.

Library modules

Introduction

The library functions for each of the different peripherals on the board will be described and an example program given. The example programs have been tested. The source code for each example is given in:

K:\WELLESLEY COPY\DESIGN & CREATIVE TECHNOLOGIES\School of Engineering\Electrical & Electronic Engineering\Resources\IARLibraries\LabBoard\LibraryExamples

The library consists of a number of separate program modules that have been compiled from separate source files. Each module can be given some acronym to describe what it does, for example:

SLCD means "Serial Liquid Crystal Display"
USART means "Universal Synchronous Asynchronous Receiver Transmitter"
RTC means "Real Time Clock"
PIR means "Pyroelectric InfraRed"
The module name will be used as a prefix for every function in the module. This sets a separate namespace for each module and states the devices it applies to. Therefore all of the functions of the SLCD module will start with the four letters, SLCD. All the functions of the USART module will start with USART and so on.

All system peripherals require some initial programming to configure the processor and peripheral devices. This is done in an "Init" function. The Init function will always be preceded by a module name for example SLCDInit(), USARTInit(), RTCInit() and so on. After initiation, then the other functions in the module can be used.

The LEDs and 7 Segment displays

Driving the light emitting diodes (LEDs) and the 7 segment displays is the easiest form of output and the place to start. The LEDs are labelled PC0 to PC7 and are adjacent to the switches at the front of the board. The LEDs and 7 Segment display use a common set of lines to the processor, so writing to the LEDs will change the 7 segment display and vice versa.

Output to the LEDS is initiated by calling LEDSInit().
The LEDs can be written to in two different ways. Firstly they are numbered from 0 to 7 on the board and can be identified by a number from 0 to 7. The functions LEDSTurnOn() and LEDSTurnOff(), both use this led number to either turn on or turn off an LED.

Secondly, a group of LEDs can be specified as a bitmask, where each bit corresponds to a specific LED. Bit 0 to LED 0, bit 1 to LED1 and bit 7 to LED 7 and so on. Turning on and turning off, can then be handled by one function LEDSSetMask(). This function turns on the LEDs that are set in the mask and turns off those that are not.

The 7 segment display displays a byte in hexadecimal number base. Output to the display is initiated by calling SEG7Init(). A number can be written to the display using SEG7WriteHex().

The header file Leds.h, contains the prototypes for the functions listed above and will be included when Labboard.h is included. Similarly, the 7 Segment functions are prototyped in 7Segment.h and this will be included when Labboard.h is included. An example of how to use these functions is shown below:

```
// File : LedsAnd7Segment.c
// Program to demonstrate the LEDS and 7 Segment displays.

#include <Labboard.h>

int main(void)
{
unsigned char Number;

LEDSInit();
LEDSClear(); // Turn off all the LEDS.
// Illuminate the leds one at a time for one second each.
for (Number=0; Number < 7; Number++ )
      {
      LEDSTurnOn(Number);
      DelayMilliSec(1000);
      LEDSTurnOff(Number);
      }

LEDSSetMask(0xF0); // Set the top 4 LEDS on.
LEDSSetMask(0); // Turn off all LEDS
LEDSSetMask(0x0F); // Set the bottom 4 leds on.
LEDSSetMask(0); // Turn off all LEDS

// 7 Segment display. Count from 0 to 255, in hex.
SEG7Init();
for (Number=0; Number < 255; Number++ )
      {
```

```
        SEG7WriteHex(Number);
        DelayMilliSec(300);
        }
}
//-------------------------------------------------------------------------------------------
```

A time delay is implemented by calling the function DelayMilliSecs(), which is also part of the library.

The buzzer

The piezoelectric buzzer on the board can be turned on, off or make a short beep. There are three functions for doing this, prototyped in Buzzer.h, which is included in Labboard.h.

```
// Beep() : Turn on the buzzer for half a second.
void Beep(void);

// BuzzerOn() : Turn on the buzzer.
void BuzzerOn(void);

// BuzzerOff() : Turn off the buzzer
void BuzzerOff(void);
```

The 20 by 4 LCD Display

The LCD display communicates with the processor using TWI which is a chip to chip serial communication protocol. The Philips implementation of TWI is called I2C. For this reason the LCD is called SLCD. The SLCD has four lines and each line can display 20 characters. Each line is called a row, numbered from 0 to 3. Each row has twenty character positions numbered from 0 to 19.

A cursor can appear at any of the character positions. The cursor can be an underline or a full character block. It can be blinking or non blinking. To write a character to the display, the cursor position must be set first and then the character written. Even if there is no cursor displayed, the cursor position still determines where the character will be displayed.

The contrast of the display can be set from 0 to 50, where 50 is the highest contrast.

The display is back illuminated and the illumination level can be set. Level 1 is the lowest, and 8 is highest.

The functions will be grouped according to their purpose.

The SLCD module has the following functions:
Firstly, the functions for overall initiation and control of the display.

```
// SLCDInit() : Initiate the SLCD system by initiating the TWI. Call this function first.
void SLCDInit(void);

// DisplayOn : Turn on the display. Until this function is called, nothing will be displayed.
void SLCDDisplayOn(void);

// SLCDDisplayOff() : Turns off the display, but what is saved in display memory is not
```

```
// lost.
void SLCDDisplayOff(void);

// SLCDClearScreen() : Clear the screen. Do at the start of your code.
void SLCDClearScreen(void);

// SLCDSetContrast();
// Value : A number between 0 and 50, where 50 is the sharpest contrast.
void SLCDSetContrast(unsigned char Value);

// SLCDSetBacklightBrightness()
// Level - Parameter between 1 and 8, where 8 is the brightest.
void SLCDSetBacklightBrightness(unsigned char Level);

// SLCDShiftDisplayLeft() :  Shifts the entire contents of the display on character position to the
// left.
void SLCDShiftDisplayLeft(void);

// SLCDShiftDisplayRight() : Shifts the entire contents of the display on character position to the
// right.
void SLCDShiftDisplayRight(void);
```

The cursor position controls where text written to the display will appear. The functions for setting the cursor position are:

```
// SLCDHomeCursor() : Set position to 0,0.
void SLCDHomeCursor(void);

// SLCDSetCursorPosition(): Set the position where the next character will be displayed.
// Invalid positions will be ignored. Cursor position is 0,0 after reset.
void SLCDSetCursorPosition(unsigned char Row,unsigned char Column);
```

Functions for controlling the type of cursor are:

```
// SLCDTurnOnUnderlineCursor() : Turns on the underline cursor.
void SLCDTurnOnUnderlineCursor(void);

// SLCDTurnOffUnderlineCursor();
void SLCDTurnOffUnderlineCursor(void);

// SLCDSetCursorLeftOneSpace() :
void SLCDSetCursorLeftOneSpace();

// SLCDSetCursorRightOneSpace() :
void SLCDSetCursorRightOneSpace();

// SLCDTurnOnBlinkingCursor();
void SLCDTurnOnBlinkingCursor(void);

// SLCDTurnOffBlinkingCursor();
void SLCDTurnOffBlinkingCursor(void);
```

Writing output to the display:

```
// SLCDWriteString() : Write a string value to the current cursor position.
// pBuffer : pointer to the string to be written. A string is a number of non zero characters terminated
// by a zero byte.
void SLCDWriteString(char *pBuffer);

// SLCDWriteBuffer() : Write a buffer  to the current cursor position.
// pBuffer : pointer to an array of containing NumBytes to the written to the display starting at the
// current cursor position. The buffer can contain non ascii characters for graphical display.
// NumBytes : The number of bytes to send to the display.
void SLCDWriteBuffer(unsigned char *pBuffer, unsigned char NumBytes);
```

Deletion functions:

```
//  SLCDBackspace(); Moves the cursor one space left and the character on the cursor is deleted.
void SLCDBackspace(void);

// SLCDClearToEOL() : Clear to end of line by overwriting with spaces.
void SLCDClearToEOL(unsigned char Row,unsigned char Col);
```

Custom characters:

```
// SLCDLoadCustomCharacter() : Specify the bit map for a custom character
// at a specified character address. The character address is the value of the byte written to the
//display.
// CharAddress : Address value between 0 and 7.
// pBitMap pointer to an array or 8 bytes containing the bitmap in row
// major order from top to bottom.
void SLCDLoadCustomCharacter(unsigned char CharAddress,unsigned char *pBitMap);
```

Miscellaneous functions:

```
// SLCDDisplayFirmwareVersion() : Displays the version of the firmware used in the LCD display.
void SLCDDisplayFirmwareVersion(void);

// SLCDDisplayI2CAddress() : Displays the TWI/I2C address of the LCD device.
void SLCDDisplayI2CAddress(void);
```

The following program uses the SLCD functions and the DelayMilliSec() function which is also in the library.

```
// File : LCDExample.c
// Program to demonstrate the SLCD4by20 code.

#include <Labboard.h>

// Define a custom graphic character as character number  7.
static unsigned char Custom[] = {0x4,0,4,0x8,0x10,0x11,0xE,0};
static unsigned char GraphicBuffer[] = {7,7,7,7};

int main( void )
{
SLCDInit();
SLCDLoadCustomCharacter(7,Custom); // Specify graphic details
```

```
SLCDClearScreen();
SLCDDisplayOn();
SLCDDisplayFirmwareVersion();
DelayMilliSec(1000); // Wait one second = 1000 millisec.
SLCDShiftDisplayRight();
DelayMilliSec(1000);
SLCDShiftDisplayLeft();
DelayMilliSec(1000);
SLCDClearScreen();
SLCDSetContrast(25); // Set half contrast
SLCDSetBacklightBrightness(1); // Set minimum back lighting.
// Write 1,2,3,4 to each of the corners.
SLCDHomeCursor(); // Move to (0,0).
SLCDTurnOnUnderlineCursor();
SLCDWriteString("1");
SLCDSetCursorPosition(0,19); // End of the first line.
SLCDWriteString("2");
SLCDSetCursorPosition(3,0); // Start of the last line.
SLCDWriteString("3");
SLCDSetCursorPosition(3,19);
SLCDWriteString("4");
SLCDSetContrast(50); // Set full contrast.
DelayMilliSec(1000);
SLCDSetBacklightBrightness(8); // Set max back light.
SLCDSetCursorPosition(1,0);
SLCDWriteString("Graphic : ");
SLCDWriteBuffer(GraphicBuffer,4); // 4 graphic chars.
DelayMilliSec(1000);
SLCDDisplayOff();
DelayMilliSec(1000);
SLCDDisplayOn();
DelayMilliSec(1000);
SLCDDisplayOff();
SLCDSetBacklightBrightness(1); // Set min back light.
return 0;
}
//-------------------------------------------------------------------------
```

The EPROM

There is an electrically erasable programmable memory device on the lab board that can store up to 32K bytes of data. It also uses the TWI bus to communicate with the processor, as does the SLCD display and colour LEDs. Any data stored in the EPROM will not be lost when the power to the board is turned off, so it is way of saving program information until it is next needed.

The memory space of the EPROM is divided into 32 byte pages from 0 to 0x7ffff. There are two methods of writing data to the EPROM memory. It can be written at individual memory addresses from 0 to 0x7FFF or can be written in blocks of up to 32 bytes using a page write. However, the page boundaries are physical ones, so that data cannot be written across page boundaries using a page write. If such an attempt is made, the writing will wrap around to the beginning of the page. Writing data to the EPROM using page writes is faster than individually addressed writes, however has this page boundary limitation.

Reading data from the EPROM does not have the same restriction. Data can be read from individual memory addresses or can be read in blocks using a sequential read. The EPROM contains an internal memory pointer which is incremented for each read operation. Data can also be read using a current read, which reads from the current internal memory pointer location. This saves having to specify the read address each time a byte is read.

The library code for using the EPROM consists of 6 functions, whose prototypes are as follows:

```
// EPROMInit() : Initiate the TWI interface to the EPRROM.
void EPROMInit(void);

// EPROMWriteByte(): Write a byte to an EPROM address.
// Address : Address to write to range from 0 to 7FFF,
// Byte : Byte to write.
// Returns : 0 for success, or the TWI error that caused failure.
unsigned char EPROMWriteByte(unsigned short Address,unsigned char Byte);

// EPROMWritePage() : Write up to 32 bytes to a page address. The EPROM's
// memory is divided into 1024, 32 byte pages. Memory writes cannot be
// accross page boundaries. If a page boundary is reached, the internal
// address counter will wrap around to the start of the page.
// Address : Address to write to range from 0 to 7FFF.
// *pBuffer : Pointer to the data buffer to write.
// Number : Number of bytes to write.
// Returns : 0 for success, or the TWI error that caused failure.
unsigned char EPROMWritePage(unsigned short Address,
                             unsigned char *pBuffer,
                             unsigned char Number);

// EPROMReadByte() : Read a byte from an EPROM address.
// This function increments the internal address counter, so the
// next read will be from the next address.
// Address : Address to read from.
// pByte : pointer to location that will contain the byte read.
// Returns : 0 for success, or the TWI error that caused failure.
unsigned char EPROMReadByte(unsigned short Address,unsigned char *pByte);

// EPROMReadCurrent() : Read the byte at the current address counter value.
// There are no page boundaries in reading.
// pByte : pointer to location that will containt the byte read.
// Returns :  0 on success, or code that caused failure.
unsigned char EPROMReadCurrent(unsigned char *pByte);

// EPROMReadSequential() : Read a buffer of bytes, starting from an address.
/// This function can read the entire contents of the EPROM in one operation.
//  pAddress : Address to write to range from 0 to 7FFF.
// *pBuffer : Pointer to the data buffer to place the read data.
// Number : Number of bytes to read.
// Returns : 0 for success, or the TWI error that caused failure.
unsigned char EPROMReadSequential(unsigned short Address,
                             unsigned char *pBuffer,
```

```
                    unsigned short Number);
```

Any program that uses these should check the return code against zero. A zero return code indicates success and non zero one failure. Unfortunately, without an understanding of the TWI bus, the error codes will not make sense. Explaining the TWI functionality is beyond the current scope of this document.

An example program that reads and writes data to the EPROM is given below.

```
// EpromExample.c
// Program to test the eprom code.

#include <Labboard.h>

static unsigned char WriteBlock[32];
static unsigned char ReadBlock[32];

//---------------------------------------------------------------------
void main( void )
{
static unsigned char rc, i;
static unsigned short Address;
static unsigned char byte;

// Initiate the LCD.
SLCDInit();
SLCDClearScreen();
SLCDHomeCursor();
SLCDDisplayOn();
SLCDWriteString("TWI EPROM Demo");

EPROMInit();

// Writing and reading byte at a time.
// Write 100 bytes of 0xAB to memory and check
// that they read back the same.
SLCDSetCursorPosition(1,0);
SLCDWriteString("Writing 100 bytes");
SLCDSetCursorPosition(2,0);
SLCDWriteString("of 0xAB");
SLCDSetCursorPosition(3,0);
rc =0;
for( Address = 0; rc == 0 && Address < 100; Address++ )
      {
      rc = EPROMWriteByte(Address,0xAB);
      if ( rc == 0 )
            {
            rc = EPROMReadByte(Address,&byte);
            if ( rc == 0 )
                  {
                  if (byte != 0xAB)
                        {
```

```
                    SLCDWriteString("Memory error.");
                    return;
                    }
                }
            else
                SLCDWriteString("ReadByte Failed");
            }
        else
            SLCDWriteString("WriteByte Failed");
        }
// Writing and reading a page (32bytes) at a time.
// Fill an array to write. and clear one for reading.
for (i=0; i<32; i++)
        {
        WriteBlock[i] = 0xBA;
        ReadBlock[i] = 0;
        }
// Write a page at address 0.
rc =  EPROMWritePage(0,WriteBlock,32);
if ( rc != 0 )
        {
        SLCDWriteString("WritePage Failed");
        return;
        }
// Read the page at address 0.
rc = EPROMReadSequential(0,ReadBlock,32);
if ( rc == 0 )
        {
        // Check memory byte by byte.
        for (Address = 0; Address < 32; Address++ )
            {
            if ( ReadBlock[Address] != WriteBlock[Address] )
                {
                SLCDWriteString("Memory error.");
                return;
                }
            }
        }
SLCDWriteString("Page 0 OK");
}
//-------------------------------------------------------
```

The USART and the printf() function

The lab board has a built in RS232 to USB converter. When a USB cable is used to connect the USB-COM output to the host computer, Windows will install the driver for another COM port. This will only occur provided the driver software for Windows has been previously installed. This has been done on all the lab computers that have the AT90USB1287 board. The specific port used, will depend on what other COM ports are currently used. The port used will be sent by the Device Manager.

To see the COM port used, right click on the Computer icon on the desktop or from the Start Menu right hand column. Click on Manage and run the Device Manager. Under the Port (COM and LPT)

will be displayed. The COM port will be labelled “USB Serial Port”. The putty program can then be used to communicate to the board through the new COM port.

The USB COM port is connected to the processor’s internal USART. Before any communication is possible the USART must be initiated with the baud, number of bits per character, parity and number of stop bits. This is done by calling the function:

```
// USARTInit() : Initiate the USART interface.
// Only for 8Mhz clock speed.
// Arguments:
// Baud - Baud rate in bits/sec.
// Allowed values: 2400,4800,9600,19200,38400. These values give
// a maximum 0.2% error in clock synchronization.
// DataBits - 5,6,7,8,9
// parity - 0 none, 1 odd, 2 even.
// StopBits - 1 or 2.
// Return values:
// 1 - Success.
// -1 Invalid baud
// -2 invalid data bits.
// -3 invalid parity
// -4 invalid stop bits.
short USARTInit(const unsigned short Baud,const unsigned char DataBits,
        const unsigned char parity,const unsigned char stopbits);
```

This prototype is given in the header file Usart.h, included in Labboard.h. Commonly used settings are:
Baud : 38400, 8 data bits, no parity, and 1 stop bit, and therefore USARTInit() is called as:

```
rc =  USARTInit(38400,8,0,1);
```

The return value is placed into an integer variable “rc”, and should be checked for equalling 1, before the other USART functions are called. To output a string to the USART call the function:

```
// USARTWriteString() : Wait for the transmitter to be ready and send a null terminated
// string of bytes.
// pString - pointer to the string to send.
void USARTWriteString(const char *pString);
```

To use the printf() function the IAR library code for the function write() must replaced by code which uses the usart. This is done by adding a file called “write.c” to the project. The write.c file is found in the same folder as the library is in; that is:

K:\WELLESLEY COPY\DESIGN & CREATIVE TECHNOLOGIES\School of Engineering\Electrical & Electronic Engineering\Resources\IARLibraries\AppBoard\Lib\write.c

This file will need to be copied to the project’s folder and added to the project.

Obtaining input from the USART is somewhat more difficult. Simply replacing the IAR library’s read() function with a function that reads from the USART and then using scanf() does not work because it exits as soon as a single byte is received. The library provides functions for sending and receiving individual bytes these are:

```
// USARTReadBuffer() : Wait for an read a byte or an error from the USART.
// The lower 5,6,7, 8 or 9 bits are the data bits depending on the
// configuration in USARTInit().
// The uppermost 3 bits if set indicate receive errors;
// Bit 15 - Parity error.
// Bit 14 - Overrun error.
// Bit 13 - Frame error.
// In case of an error the lower data bits actually read are still returned.
short USARTReadBuffer(void);
```

This function will not return until a byte has been received or an error has occurred. Therefore, since a program must generally poll other devices as well as the USART, a function for checking whether there is a byte ready to receive is provided:

```
// USARTReceiveReady() Tests for whether  the receive buffer contains data
// to be read.
// Returns 1 if there is and 0 if not.
short USARTReceiveReady(void);
```

The byte level functions for writing to the USART are the converse of the above:

```
// USARTTransmitReady() : Tests for whether the transmit buffer is ready
// to accept new data.
// Returns 1 if there is and 0 if not.
short USARTTransmitReady(void);

// USARTWriteBuffer() : Wait for transmitter ready and send a byte.
// Data - The data to send. Will send 9 bit data if configured in USARTInit().
// Waits until transmitter is ready, but not until all the data has been sent.
void USARTWriteBuffer(const unsigned short Data);
```

To receive to string the USARTReadBuffer() function needs to be called iteratively to fill an array. This is done by the function ReadString() is the example program below. Each time a byte is received, it is written back to the USART, so that it will be seen on the terminal screen. When a carriage return is received (‘\r’ in C), the null byte is written to the end of the receive array.

After a string has been read, it can be converted into a integer or floating point number using sscanf() and atof() respectively. The C library function sscanf() is like the function scanf() except reads from a string instead of the standard input. The author was unable to get sscanf() to convert a string to a floating point number, consequently had to use the function atof().

When writing data output to the terminal using the USART, both a carriage return (‘\r’) and line feed characters (‘\n’) need to be sent to move the terminal cursor to a new line. This is because carriage return only moves the cursor to the start of a line and line feed drops the cursor down a line, scrolling the display if necessary.

The example program reads in an integer and a floating point number and prints them out using printf().

```
// UsartExample.c
```

```
#include <Labboard.h>
#include <stdio.h>
#include <stdlib.h>  // for the atof() prototype.

#define BUFFERSIZE      20

int ReadString(char *pBuffer, int size);

void main(void)
{
static int rc, Number;
static float val;
char NumberBuffer[BUFFERSIZE];

// 38400 baurd,8 data bits,  no parity, 1 stop bit.
rc =  USARTInit(38400,8,0,1);
if ( rc == 1 )
      {
      USARTWriteString("Demonstrating how to use the Usart functions.\n\r");
      USARTWriteString("with printf \n\r" );
      for ( ;; )
            {
            printf("Enter a integer : ");
            ReadString(NumberBuffer,(BUFFERSIZE-1));
            sscanf(NumberBuffer,"%d",&Number);
            printf("\n\rInteger = %d\r\n",Number);

            printf("Enter a floating point number : ");
            ReadString(NumberBuffer,(BUFFERSIZE-1));
            val = atof(NumberBuffer);
            printf("\n\rDouble = %lf\n\r",val);
            }
      }
}
//---------------------------------------------------------------------
// ReadString() : Read the received characters into the buffer passed
// and echo each character received.
// pBuffer : pointer to a buffer to receive the bytes.
// size : The max number of bytes to receive.
int ReadString(char *pBuffer, int size)
{
short rc, count;

count = 0;
rc = USARTReadBuffer();
while (size > 0 && (rc&0xFF00) == 0 && (rc & 0xFF) != '\r' )
      {
      USARTWriteBuffer(rc); // Echo
      *pBuffer = (char)(rc&0xFF); // Put the lower byte in.
      pBuffer++;
      size--;
      count++;
```

```
        rc = USARTReadBuffer();
        }
*pBuffer = 0; // Terminate the string.
return count;
}
//---------------------------------------------------------------------
```

The toggle switches and buttons

There are two types of switch on the board. Firstly there are the 8 toggle switches which can either be up or down and toggle from one state to the other. Secondly there are the three button switches that are momentarily depressed. A program wants to take some action when a switch is either down or up. The client or user interface for the switches is implemented in the library by using *callback* functions. As the name implies, a callback function is a function called by the library when some event occurs. Callback functions are used for many of the devices interfaces in the library.

The name of a function in the C language is the program address or pointer to of its code. Therefore the name of a function is a pointer to that function and can be passed as an argument to a library function. The library code for the toggle switches has an initiation function that takes as an argument the name of the function that will be called whenever the state of one of the toggle switches changes. This callback function is passed two argument when it is called. Firstly is the number of the switch (0-7) and secondly is an integer 0 or 1, depending on whether the switch is down or up respectively. A toggle callback function will have the prototype:

```
void ToggleCallback(unsigned char SwitchNumber, unsigned char DownUp);
```

The toggle switch initiation function has the prototype:

```
void TOGGLEInit(pfTOGGLEEVENT epf);
```

The type named pfTOGGLEEVENT is function pointer to a ToggleCallback function. To initiate the toggles so that the callback is made, pass the ToggleCallback function name as the argument to the TOGGLEInit() function as:

```
TOGGLEInit(ToggleCallback);
```

Finally, the state of the switches needs to be regularly read so that the library can call the callback function. This is done by calling the function TOGGLEPoll(void), in a C *while* or *for* iteration. The TOGGLEPoll() function removes any switch bouncing before calling the callback function. An example program is provided below, after the button interface has been described.

The buttons use a similar but slightly different callback interface. For each button, there are two events, either press or release. Since there are three buttons there are six possible button events: PB1_PRESS,PB1_RELEASE, PB2_PRESS,PB2_RELEASE, PB3_PRESS,PB3_RELEASE. The library header file Buttons.h defines these events using the C enumeration type as:

```
typedef enum {PB1_PRESS,PB1_RELEASE,PB2_PRESS,PB2_RELEASE,PB3_PRESS,
    PB3_RELEASE} BUTTON_EVENT;
```

The button callback function passes one of these events as an argument. Therefore the prototype for the button callback function is:

```
void (*pfButtonEvent)(BUTTON_EVENT e);
```

The BUTTONInit() function takes as an argument, a button callback function and has the prototype:

```
void BUTTONInit(void (*pButtonEvent)(BUTTON_EVENT e));
```

Finally, the state of the buttons is read using the BOTTONPoll() function. This function removes switch bouncing before making a callback with an event. The following is an example of using the toggle switches and the buttons:

```
// TogglesAndButtons.c
// Program to demonstrate the buttons and switches.

#include <Labboard.h>

// Prototypes for the event handlers.
void TogglesState(unsigned char tid,unsigned char level);
void ButtonState(BUTTON_EVENT be);

// Outputs the value of the toggle switches and buttons.
void main( void )
{
TOGGLEInit(TogglesState);
BUTTONInit(ButtonState);

while ( 1 )
	{
	TOGGLEPoll();
	BUTTONPoll();
	}
}
//---------------------------------------------------------------------------
void TogglesState(unsigned char tid,unsigned char level)
{
if ( level == 1 )
	LEDSTurnOn(tid);
else
	LEDSTurnOff(tid);
}
//---------------------------------------------------------------------------
void ButtonState(BUTTON_EVENT be)
{
switch ( be )
	{
	case PB1_PRESS :	LEDSTurnOn(0); break;
	case PB1_RELEASE : LEDSTurnOff(0); break;
	case PB2_PRESS :	LEDSTurnOn(1); break;
	case PB2_RELEASE : LEDSTurnOff(1); break;
	case PB3_PRESS :	LEDSTurnOn(2); break;
	case PB3_RELEASE : LEDSTurnOff(2); break;
	}
}
```

//---

The keypad

The keypad has twelve keys with ASCII equivalent values from. 0 to 9,* and #. The callback function for this device passes the ASCII value of the key pressed. In a large object oriented program the object that receives the keypad event will change during program execution. To make this possible, the callback handler passes a void pointer. This pointer value can be set to zero when not needed. Therefore, the callback function for keypad events has the prototype:

```
void (*pfKeyEvent)(void *pObj,unsigned char Value);
```

Where pObj is a previously assigned pointer,. and Value is the ASCII value of the key pressed. Any switch bouncing is removed by the key polling function KEYPADPoll().

The use of the key pad is initiated by calling KEYPADInit() which has the prototype:

```
void KEYPADInit(void (*pfKeyEvent)(void *pObj,unsigned char Value),
            void *pObj);
```

The first argument is the name of the callback function, and the second is an arbitrary void pointer which will be passed in every callback. The callback function or the void pointer can be changed using the KEYPADSetHandler() function, which has the prototype:

```
void KEYPADSetHandler(void (*pfKeyEvent)(void *pObj,unsigned char Value),
                                    void *pObj);
```

The keypad is polled and any switch bouncing removed by calling the function:

```
void KEYPADPoll(void);
```

An example program that displays the key pressed on the LCD display is shown below.

```
// KeypadExample.c
#include <Labboard.h>

// Program echoes the keypad presses to the LCD display.
// The callback function
static void KeyPressed(void *pob,unsigned char k);


void main(void )
{
// Initiate the LCD.
SLCDInit();
SLCDClearScreen();
SLCDHomeCursor();
SLCDDisplayOn();
// Initiate the keypad.
KEYPADInit(KeyPressed,0);
for (;;)
      {
```

```
        KEYPADPoll();
        }
}
//---------------------------------------------------------------------
static void KeyPressed(void *pobj,unsigned char key)
{
static int CurX; // Current cursor position.

SLCDSetCursorPosition(0,CurX);
SLCDWriteBuffer(&key,1) ;
CurX++;
if ( CurX == 19 )
      CurX = 0;
}
//---------------------------------------------------------------------
```

The TSWB switches and rotary encoder

Just above the toggle switches is a round black device which consists of 5 switches and a central rotary encoder. It will be called the TSWB encoder in this document. The switches are at the top, sides, bottom and centre.

The inner circular disk with raised black dots is a rotary encoder. Turning it clockwise or anticlockwise produces rectangular pulses to two of the interrupt inputs to the processor. It can produce 12 pulses per one revolution and the direction of rotation can be determined. When you write your own code for this device, be aware that it the signals to the processor are not quadrature encoded. That is, the two signals are not separated in phase by 90 degrees.

A quadrature encoder requires a software state machine to decode direction and count, however the hardware in this device has a clever system that makes the software much simpler. *When turned clockwise, the rising edges of the two signals are simultaneous. When turned anticlockwise, the falling edges of both signals are simultaneous. The turn count is increased for each rising edge of either signal or decreased for each falling edge of either signal.* There will be 12 counts per revolution when measurements are down in this way.

From a software perspective, TSWB encoder consists of two separate devices in the one case, the switch and the rotary encoder. The rotary encoder functions will be described first. The client's code defines a callback function that will be called whenever there is a change in count or in direction of the rotary encoder. The callback function has the form:

```
void (*pfIncrement)(signed char Value, unsigned char Direction);
```

Where 'Value' is the current rotation count of the encoder and Direction is 1 for clockwise rotation of 0 for anticlockwise rotation. This callback function is called by the interrupt subroutines for each of the signal channels, so any global variables changed by this function must be of the volatile type. The callback function is passed to the REInit() function to initiate the rotary encoder part of the device. REInit() has the prototype:

```
void REInit(void (*pfIncrement)(signed char Value, unsigned char Direction));
```

At any time, the current counter and direction values can be found by calling:

```
// REGetCounter() : Return the current counter value.
short REGetCounter(void);

// REGetDirection() : Return the current direction. Positive is clockwise
// Zero is anticlockwise
unsigned char REGetDirection(void);
```

Finally, to stop the callbacks from occurring, call REStop(), which has the following prototype.

```
// REStop() : Stop the callbacks from the rotary encoder.
void REStop(void);
```

The TSWB switches are handled in similar way to the buttons. An enumerated event is defined for the press and release of each of the switches. These are given the names:

TSWB_LEFT : Event for the left switch being depressed and released.
TSWB_CENTRE : : Event for the centre switch being depressed and released
TSWB_DOWN : : Event for the down switch being depressed and released
TSWB_RIGHT : Event for the right switch being depressed and released
TSWB_UP : Event for the up switch being depressed and released.

The client's code is notified of a switch event by a callback function with the prototype:

```
void (*pfTSWBEvent)(TSWB_EVENT e));
```

The callback function is called whenever one of the above switch events occurs and the event is passed to the client's callback function. The TSWB system is initiated by calling:

```
void TSWBInit(void (*pfTSWBEvent)(TSWB_EVENT e));
```

After the TSWBInit() function has been called, switch polling function needs to be called in an iterative loop. This TSWBPoll():

```
void TSWBPoll(void);
```

When the poll function detects switch events, they will be passed to the callback function. An example program using the TSWB switches and rotary encoder is given below.

```
// TSWBSwitchesAndEncoder.c
//Demonstrating TSWB switches and encoder.
#include <Labboard.h>
#include <string.h>
#include <stdio.h>

// Callback functions
void SwitchEvent(TSWB_EVENT e);
void RECallback(signed char Value, unsigned char Direction);

// Global variables.
unsigned char REEvent;
signed char REVal;
```

```
unsigned char REDir;

void main( void )
{
char Buffer[20];

TSWBInit(SwitchEvent);
REInit(RECallback);

SLCDInit();
SLCDClearScreen();
SLCDHomeCursor();
SLCDDisplayOn();
SLCDWriteString("TSWB Demo");

for (;;)
	{
	TSWBPoll(); // wait for an event.
	if ( REEvent == 1 )
		{
		REEvent = 0;
		sprintf(Buffer,"Val= %d, Dir = %d",REVal,REDir);
		SLCDClearToEOL(1,0);
		SLCDSetCursorPosition(1,0);
		SLCDWriteString(Buffer);
		}
	}
}
//---------------------------------------------------------------------
void SwitchEvent(TSWB_EVENT e)
{
SLCDClearToEOL(2,0);
SLCDSetCursorPosition(2,0);

switch ( e )
	{
	case TSWB_LEFT : SLCDWriteString("Left."); break;
	case TSWB_CENTRE : SLCDWriteString("Centre."); break;
	case TSWB_DOWN : SLCDWriteString("Down."); break;
	case TSWB_RIGHT : SLCDWriteString("Right."); break;
	case TSWB_UP : SLCDWriteString("Up."); break;
	}
}
//---------------------------------------------------------------------
void RECallback(signed char Value, unsigned char Direction)
{
REEvent = 1;
REVal = Value;
REDir = Direction;
}
//---------------------------------------------------------------------
```

The fan motor and speed measurement

The motor that drives the fan is driven by a pulse width modulated signal at 20 Khz. The percentage of the pulse cycle time for which the output is high is called the percent duty. This can be set by calling the function:

```
// MotorPWM() : Drive the motor with 20 Khz, PWM of a specified duty
// cycle.
void MotorPWM(unsigned char Percent);
```

The motor can be turned on or off using the functions:

```
// Turn motor output on.
void MotorON(void);

// Turn motor output off.
void MotorOFF(void);
```

The motor drives the fan. The fan has two small cutouts in its casing. When these cutouts pass an infrared sensor a pulse is generated. Since there are two cutouts, there are two pulses per fan revolution. By counting the number of pulses per second, the fan speed can be measured. The speed is converted into revs per minute and can be obtained by calling the function:

```
// FanSpeedRPM() : Measure the fan speed in RPM.
unsigned short FanSpeedRPM(void);
```

There is a short variable delay in calling this function, in order for the measurement to be made. It does not reliably measure very slow speeds.

A program demonstrating driving the motor and measuring the speed is shown next.

```
// MotorSpeed.c
// Program to demonstrate using the motor and speed measurement.

#include <Labboard.h>
#include <stdio.h>  // for the sprintf() prototype.

int main( void )
{
unsigned char DutyCycle;
static char MsgBuf[21];
unsigned short Speed;

SLCDInit();
SLCDClearScreen();
SLCDHomeCursor();
SLCDDisplayOn();
SLCDWriteString("Motor speed demo");

for ( DutyCycle = 10; DutyCycle <=30; DutyCycle += 10 )
        {
         MotorPWM(DutyCycle);
```

```
            MotorON();
            sprintf(MsgBuf,"Duty: %d percent",DutyCycle);
            SLCDClearToEOL(1,0);
            SLCDSetCursorPosition(1,0);
            SLCDWriteString(MsgBuf);
            MotorPWM(DutyCycle);
   // Wait half second for speed to stabilize.
            DelayMilliSec(500);
            SLCDClearToEOL(2,0);
            SLCDSetCursorPosition(2,0);
            SLCDWriteString("Measuring speed...");
            Speed = FanSpeedRPM();
            sprintf(MsgBuf,"Speed: %d RPM",Speed);
            SLCDClearToEOL(2,0);
            SLCDSetCursorPosition(2,0);
            SLCDWriteString(MsgBuf);
  // Wait while showing speed.
            DelayMilliSec(2000);
            }
for ( DutyCycle = 30; DutyCycle > 0; DutyCycle -= 10 )
            {
            MotorPWM(DutyCycle);
            MotorON();
            sprintf(MsgBuf,"Duty: %d percent",DutyCycle);
            SLCDClearToEOL(1,0);
            SLCDSetCursorPosition(1,0);
            SLCDWriteString(MsgBuf);
            MotorPWM(DutyCycle);
 // Wait a half second for speed to stabilize.
            DelayMilliSec(500);
            Speed = FanSpeedRPM();
            SLCDClearToEOL(2,0);
            SLCDSetCursorPosition(2,0);
            SLCDWriteString("Measuring speed...");
            sprintf(MsgBuf,"Speed: %d RPM",Speed);
            SLCDClearToEOL(2,0);
            SLCDSetCursorPosition(2,0);
            SLCDWriteString(MsgBuf);
            DelayMilliSec(2000);
            }
MotorOFF();
SLCDDisplayOff();
}
//---------------------------------------------------------------------
```

The speaker

The speaker is driven from a filtered output from the processor. The frequency of the speaker output can be set between 61 and 15625Hz. The speaker can be turned on or turned off:

```
// SpeakerOn() : Turn on the speaker by enabling the OC2A output.
void SpeakerOn(void);
```

```
// SpeakerOff() : Turn off the speaker by disabling the OC2A output.
void SpeakerOff(void);

//  SpeakerSetFrequency() : Set the speaker frequency. Very approximate.
// Max is 15625Hz, min is 61 Hz.
void SpeakerSetFrequency(unsigned short Frequency);
```

The MicAndSpeaker.c program under the microphone section below, gives an example of using the speaker output.

The analog input devices

There are four analog to digital converter inputs to the processor. These channels are used as follows:

ADC0:. The light level sensor, which is the LDR close to the lamp.
ADC1: A toggle switch at the front of the board selects the input to this either the relative humidity sensor or to potentiometer 2.
ADC2: A toggle switch at the front of the board selects the input to either the microphone or the potentiometer 1.
ADC3: The temperature sensor for measuring the temperature of the PWM heated resistor.

The library code for using these inputs will now be described. In general the ADC results can be measured using the ADC interface whose prototypes are in the header file adc.h. This header file is included in Labboard.h. There are three functions whose prototypes are as follows:

```
// ADCInit() : Initiate the ADC. Must be called first before any conversions.
void ADCInit(void);

// ADCConvert() : Perform and ADC conversion on a single channel and return
// the 10 bit result. Waits until conversion is complete.
unsigned short ADCSingleConvert(unsigned char Ch);
```

The ADC conversions are all done in 10 bits. Since the voltage reference to the ADC converter is 5 volts, a measured conversion value of 1024 corresponds to a voltage input of 5 volts. To convert the measured ADC value to a string which contains the voltage expressed to 3 decimal places, use the following function:

```
// ADCReadingToVolts() : Return a pointer to a string containing the ADC 10 bit
// reading from 0- 5 volts.
// Reading - ADC value to convert.
// pOuts : pointer to 7 byte buffer to receive result.
void ADCReadingToVolts(unsigned short Reading,char *pOuts);
```

A demonstration program that reads the voltage values from the two potentiometers is given below. To use the program, ensure that the pots are selected on the two toggle switches. Secondly, please note that the voltage output from pot 2, increases as the pot is turned clockwise, but the voltage output from pot 1 *decreases* as the pot is turned clockwise. If the pot2 toggle switch changed to select RH, the output from the relative humidity sensor will be shown.

```
// Potentiometers.c
// Program to demonstrate how read the ADC single conversions.
#include <Labboard.h>
```

```
#include <stdio.h>
#include <string.h>

void  main( void )
{
unsigned short Pot1, Pot2;
char Buffer[10];

SLCDInit();
SLCDClearScreen();
SLCDHomeCursor();
SLCDDisplayOn();

ADCInit();
SLCDWriteString("ADC POT1 POT2 Test");
SLCDSetCursorPosition(1,0);
SLCDWriteString("Read every 250ms.");
while ( 1 )
      {
      // Clear the line of any previous text.
      SLCDClearToEOL(2,0);
      SLCDSetCursorPosition(2,0);

      // Read and display pot 1, channel 2.
      Pot1 =  ADCSingleConvert(2);
      SLCDSetCursorPosition(2,0);
      SLCDWriteString("Pot1 : ");
      ADCReadingToVolts(Pot1,Buffer);
      SLCDWriteString(Buffer);
      SLCDWriteString(" Volts");

      // Clear the line of any previous text.
      SLCDClearToEOL(3,0);
      SLCDSetCursorPosition(3,0);

      // Read and display pot 2, channel 1.
      Pot2 =  ADCSingleConvert(1);
      SLCDWriteString("Pot2 : ");
      ADCReadingToVolts(Pot2,Buffer);
      SLCDWriteString(Buffer);
      SLCDWriteString(" Volts");

      // Wait for the next conversion time.
      DelayMilliSec(250);
      }
}
//---------------------------------------------------------------------------
```

The microphone

The microphone input and its signal conditioning circuitry is designed so that a short sharp sound such as a clap of finger snap can be detected. When the ADC value rises above a threshold value a

clap has been detected. To detect such a short pulse requires continuous ADC conversions and the use of interrupts. This code has been incorporated into the microphone module of the library.

The client program provides a callback function that the library will call if a pulse from the microphone has been detected. The client's callback function must have the prototype:

```
void MicPulse(void) ;
```

The name of this function is passed to the MICInit() function of the library:

```
// MICInit() : Initiate the microphone.
// pfMicCB : The callback function called.
void MICInit(void (*pfMicCB)(void));
```

While waiting for the callback, the client's program calls the function MICPoll(), whose prototype is:

```
// MICPoll() : Poll the microphone system. The call to the client's callback function is made by
// this function.
void MICPoll(void);
```

When the microphone system is no longer needed, the function MICFinish() is called:

```
// MICFinish(): Terminate the microphone detection.
void MICFinish(void);
```

The program below uses the speaker to create a sound. When it is detected by the microphone, the callback function is called, causing the waiting loop in the main program to exit.

```
// MicAndSpeaker : Program to create short
// tone from the speaker and read it using the microphone.

#include <Labboard.h>

#define DETECTFREQ      200     // Speaker frequency.

static void MicFound(void); // The callback function.
static  unsigned char Found;

void main( void )
{
SLCDInit();
SLCDClearScreen();
SLCDHomeCursor();
SLCDDisplayOn();
MICInit(MicFound);

Found = 0;
SpeakerSetFrequency(DETECTFREQ);
SpeakerOn();
SLCDWriteString("Microphone test");
SLCDSetCursorPosition(1,0);
SLCDWriteString("Listening...");
```

```
while (Found == 0 )
	{
	MICPoll();
	}
MICFinish();
SpeakerOff();
SLCDSetCursorPosition(2,0);
SLCDWriteString("Mic found");
SLCDSetCursorPosition(3,0);
SLCDWriteString("Terminating.");
}
//---------------------------------------------------------------------
// MicFound() : Callback from the mic interface.
static void MicFound(void)
{
Found = 1;
}
//---------------------------------------------------------------------
```

The lamp and LDR sensor

There is an incandescent lamp that can be illuminated by a PWM output from the processor in a similar manner to the motor and heater. The light level is measured by a light dependent resistor (LDR) below the LDR. The library functions for driving the lamp and reading the light level are:

```
// LampOn() : Turn the Lamp on with a specified  PWM duty cycle.
void LampOn(unsigned char Percent);

// LampOff() : turn the lamp off.
void LampOff(void);

// LightLevel() : Read the light level. The return value changes from 0 to 1024 as the sensor output voltage changes from 0- 5 volt.
unsigned short LightLevel(void);
```

The heater and temperature measurement

A resistor below the motor fan can be heated by driving it with a PWM output from the processor. A temperature sensor is fastened to the resistor so that its temperature can be read. The library function for heating the resistor and measuring its temperature are:

```
// HeaterOn() : Turn the heater on with a specified  PWM duty cycle.
void HeaterOn(unsigned char Percent);

// HeaterOff() : turn the heater off.
void HeaterOff(void);

// Temperature() : Read the temp sensor. The return value changes from 0- 1024 as the sensor output voltage changes from 0- 5 volt.
unsigned short Temperature(void);
```

The RGB LED

RGB stands for Red, Green Blue. The Cat3626 RGB led can output three different colours of various intensities at the same time, producing different resulting colours of light. It consists of separate red, green and blue LEDs which can be driven with different currents, from 0.5 milliamps to 32 milliamps each. The current through each component LED can be set and read. Furthermore each component LED and be enabled or disabled.

The Cat3626 device is connected to the processor via the TWI/I2C bus. The bus and the device are initiated before use by calling the function:

```
// RGBInit() :  Initiate the led driver. Initiates the TWI.
void RGBInit(void);
```

Each of the component LEDs are identified by a defined macro, RGB_RED, RGB_GREEN, RGB_BLUE. Each of these macros specifies different bits. They can be combined by bit ORing them together, for example:

```
RGB_RED| RGB_BLUE   specifies both red and blue.
RGB_RED| RGB_BLUE|RGB_GREEN   specifies all three leds.
```

The following two functions are passed a bit mask that consists of bit OR of the macros together to enable or disable the component LEDS.

```
// RGBEnable() : Enable the specified LEDs. One or more of the defines
// above can be bit or-red together. eg LEDA1|LEDA2.
unsigned char RGBEnable(unsigned char LED_ID);

// RGBDisable() : Disable the specified LEDs. One or more of the defines
// above can be bit or-red together. eg LEDA1|LEDA2.
unsigned  char RGBDisable(unsigned char LedID);
```

The current through the component LEDs are set through the following functions:

```
// RGBWriteRedCurrent(unsigned char MilliAmps). A Milliamps of zero
// produces 0.5ma current.
unsigned char RGBWriteRedCurrent(unsigned char MilliAmps);

// RGBCWriteGreenCurrent(unsigned char MilliAmps). A Milliamps of zero
// produces 0.5ma current.
unsigned char RGBWriteGreenCurrent(unsigned char MilliAmps);

// RGBCWriteBlueCurent(unsigned char MilliAmps)
unsigned char RGBWriteBlueCurrent(unsigned char MilliAmps);
```

Each of the above functions returns zero if successful or the TWI error code if not. Any error should be reported to the author. The most likely cause will be a faulty board.

The currents through the LEDs can be read using the following functions:

```
// RGBReadRedCurrent(unsigned char *pCurrent)
unsigned char RGBReadRedCurrent(unsigned char *pCurrent);
```

```
// RGBReadGreenCurrent(unsigned char *pCurrent).  A milliamps of zero
// produces 0.5ma current.
unsigned char RGBReadGreenCurrent(unsigned char *pCurrent);

// RGBReadBlue(unsigned char *pCurrent)
unsigned char RGBReadBlueCurrent(unsigned char *pCurrent);
```

An example program is as follows:

```
// RGBLed.c
// Demo program for driving the RGB leds.

#include <Labboard.h>
#include <stdio.h>  // for the sprintf() prototype.

void main(void)
{
unsigned char RedCurrent,BlueCurrent,GreenCurrent;
char Buffer[21];  //LCD 20 chars wide plus one for the null.

SLCDInit();
SLCDClearScreen();
SLCDHomeCursor();
SLCDDisplayOn();
SLCDWriteString("RGB LED demo");
RGBInit();

RGBEnable(RGB_RED);
RGBWriteRedCurrent(2);
RGBReadRedCurrent(&RedCurrent);
SLCDSetCursorPosition(1,0);
sprintf(Buffer,"Red Current = %d ma",RedCurrent);
SLCDWriteString(Buffer);
DelayMilliSec(2000);

// Red and green
RGBEnable(RGB_RED|RGB_GREEN);
RGBWriteGreenCurrent(2);
RGBReadGreenCurrent(&GreenCurrent);
SLCDSetCursorPosition(2,0);
sprintf(Buffer,"Green Current = %d ma",GreenCurrent);
SLCDWriteString(Buffer);
DelayMilliSec(2000);

// Red, green and blue.
RGBEnable(RGB_RED|RGB_GREEN|RGB_BLUE);
RGBWriteBlueCurrent(2);
RGBReadBlueCurrent(&BlueCurrent);
SLCDSetCursorPosition(3,0);
sprintf(Buffer,"Blue Current = %d ma",BlueCurrent);
SLCDWriteString(Buffer);
DelayMilliSec(2000);
```

```
// Disable all leds.
RGBDisable(RGB_RED);
RGBDisable(RGB_GREEN);
RGBDisable(RGB_BLUE);
}
//---------------------------------------------------------------------
```

The Real Time Clock

The real time clock keeps track of changes in the time and date. The time and date are initially set programmatically. The library code enables the time and date to be set and read. In addition it provides a facility for a program function to be called after every second. Finally it can provided battery backup ram which can be used to save program data, which will not be lost when the power is off. However to maintain the data, a battery input to power the chip. There is not a battery facility on the lab boards.

The RTC device is initiated and a callback function set for callback after every second, using RTCInit():

```
void RTCInit(void (*pfSecondsAlarm)(void));
```

If no seconds callback is required, then the argument is zero. The callback function passed is called from an interrupt subroutine. Therefore the callback function should be short and should only set the values of volatile variables. In the demonstration program below, only a flag variable is set, which the main program reads.

The time and date data are saved and restored to a DATETIME structure, defined in RTC.h and reproduced here:

```
typedef struct  {
        unsigned char Seconds;
        unsigned char Minutes;
        unsigned char Hours;  // 24 hour representation.
        unsigned char Day;      // Day of week.
        unsigned char DayOfMonth;
        unsigned char Month;
        unsigned char Year;  // Two digit.
        } DATETIME;
typedef DATETIME *pDATETIME;
```

When setting an initial time and date, the fields in a structure variable of this type are filled out and then the address of the DATETIME structure is passed to the function:

```
void RTCSet(pDATETIME pdt);
```

After this has been done, the RTC will keep track of changes in time and date. At some later time, the current time and date can be read into a DATETIME structure variable using:

```
void RTCGet(pDATETIME pdt);
```

The structure variable whose address is passed in the argument “pdt”, will be filled with the current values of the variables from the RTC device.

To enable a regular callback at one second intervals, the function RTCSecondsAlarm() is called. The seconds alarm can be disabled using RTCDisableAlarm().

An example program showing how to implement a simple clock using the LCD display for output is shown below.

```
// RTClock.c
// Demonstration program for the Real Time Clock.

#include <Labboard.h>
#include <stdio.h>

void SecsAlarm(void);
void ShowTime(void);

volatile unsigned char SecondExpired;

void main(void)
{
static DATETIME ToSet;
char Buffer[21];

SLCDInit();
SLCDClearScreen();
SLCDHomeCursor();
SLCDDisplayOn();
SLCDWriteString("Real Time Clock");

// Initiate the RTC and set the seconds callback.
RTCInit(SecsAlarm);
// The structure current contains the setting read.
ToSet.Year =  13;
ToSet.Month = 3;
ToSet.DayOfMonth = 14;
ToSet.Day = 4;
ToSet.Hours = 11;
ToSet.Minutes = 55;
ToSet.Seconds = 40;
RTCSet(&ToSet);  //Set the date and time.
ShowTime();
SecondExpired = 0;
RTCSecondsAlarm(); // Enable the alarm.
for (;; )
        {
        while ( SecondExpired == 0 )
                ; // Wait for a second to expire.
        ShowTime();
        SecondExpired = 0;
        }
```

```
}
//---------------------------------------------------------------------
// SecsAlarm() : This is an callback from an interrupt subroutine.
void SecsAlarm(void)
{
SecondExpired = 1;
}
//---------------------------------------------------------------------
// ShowTime() : Read and show the current time.
void ShowTime(void)
{
static char Buffer[21];
static DATETIME ToGet;
      // Display it.
RTCGet(&ToGet);
SLCDClearToEOL(1,0);
SLCDSetCursorPosition(1,0);
sprintf(Buffer,"%2d:%2d:%2d",ToGet.Hours,ToGet.Minutes,ToGet.Seconds);
SLCDWriteString(Buffer);
SLCDClearToEOL(2,0);
SLCDSetCursorPosition(2,0);
sprintf(Buffer,"%2d/%2d/%2d",ToGet.Day,ToGet.Month,ToGet.Year);
SLCDWriteString(Buffer);
}
//---------------------------------------------------------------------
```

The frequency generator and frequency measurement

The boards have frequency generator whose frequency output can be selected by pressing the FR-SEL button. The number on the FRQ_NUM , 7 segment display corresponds to an output frequency. The correspondence is as shown in the table below:

Generator Setting Displayed on 7 Segment display	Frequency Hz	Duty Cycle %
0	Off	
1	1	50
2	10	50
3	100	50
4	500	50
5	1K	50
6	2K	50
7	5K	50
8	10K	50
9	20K	50
10	100K	50
11	1K	10
12	1K	30
13	1K	60
14	1K	80
15	1K	90

This frequency is fed into the ICP1 input on the processor. The ICP input enabled the frequency to be measured by measuring the time between signal edges. The library function ICPGetFrequency() does this. It does all the required initiation before making the measurement. It's prototype is :

```
// ICPGetFrequency() : Returns the measured frequency as a 32bit value.
// Uses the time difference between positive edges of the input signal.
unsigned long ICPGetFrequency(void);
```

The following is a demonstration program that uses this function. A frequency measurement is made every time the user presses one of the front buttons PB1,PB2,PB3.

```
// FrequencyMeasureExample.c
// Demonstrating how to use frequency measurement.

#include <Labboard.h>
#include <stdio.h>

// Program performs a frequency measurement every time any
// button is pressed and displays the frequency.

void ButtonState(BUTTON_EVENT be);

void main(void)
{
SLCDInit();
BUTTONInit(ButtonState);

SLCDClearScreen();
SLCDHomeCursor();
SLCDDisplayOn();
SLCDWriteString("Freq. Measure.");

SLCDSetCursorPosition(1,0);
SLCDWriteString("Press any button.");
// Wait until a button is pressed.
while ( 1 )
      {
      BUTTONPoll();
      }
}
//-------------------------------------------------------------
void ButtonState(BUTTON_EVENT be)
{
unsigned long  Frequency;
char Buffer[21];

switch ( be )
      {
      case PB1_PRESS :
      case PB2_PRESS :
      case PB3_PRESS :
```

```
            Frequency = ICPGetFrequency();
            SLCDClearToEOL(2,0);
            SLCDSetCursorPosition(2,0);
            sprintf(Buffer,"Freq = %ld Hz.",Frequency);
            SLCDWriteString(Buffer);
            break;
      default :
            break; // dont care about button releases.
      }
}
//------------------------------------------------------------------
```

Conclusion

The library functions for devices on the AT90USB1287 lab board have been presented and for most devices examples programs have been given. The processor's USB interface and the external 20 pin header have not been included. The separate library for USB use is still under development.

There are many other functions in the library that are called by the modules described, for example the TWI interface code. The header files detailing what these functions do are in the include folder. All the header files for the functions in the library are included when the labboard.h is included in a source file. Therefore, any of the functions not mentioned in this document but in the header files can be used in your programs.